

yt-agent2

Current File Contents

June 23, 2026

Files in this document

- 01. agents/transcript_agent.py
- 02. agents/blog_post_agent.py
- 03. orchestrator.py
- 04. app.py
- 05. requirements.txt
- 06. railway.toml
- 07. nixpacks.toml

agents/transcript_agent.py

yt-agent2/agents/transcript_agent.py

```

"""
TranscriptAgent
-----
Responsibility: Given a YouTube URL, download the audio and return
a plain-text transcript using Groq's Whisper API.

Dependencies: yt-dlp, groq, python-dotenv
"""

import os
import tempfile
import yt_dlp
from groq import Groq
from dotenv import load_dotenv

load_dotenv()

class TranscriptAgent:
    """
    Agent 1 – Downloads YouTube audio and transcribes it via Groq Whisper.

    Usage:
        agent = TranscriptAgent()
        transcript = agent.run("https://www.youtube.com/watch?v=...")
    """

    # Groq Whisper supports files up to 25 MB.
    # yt-dlp will pick the smallest audio stream to stay under this.
    MAX_FILE_SIZE_MB = 24

    def __init__(self):
        api_key = os.getenv("GROQ_API_KEY")
        if not api_key:
            raise EnvironmentError(
                "GROQ_API_KEY not found. Add it to your .env file."
            )
        self.client = Groq(api_key=api_key)
        print("[TranscriptAgent] Initialised")

    # -----
    # Public interface
    # -----

    def run(self, youtube_url: str) -> str:
        """
        Main entry point called by the Orchestrator.

        Args:
            youtube_url: Any valid YouTube video URL.

        Returns:
            Plain-text transcript string.

        Raises:
            ValueError: If the URL is empty or obviously invalid.
            RuntimeError: If download or transcription fails.
        """
        self._validate_url(youtube_url)

        print(f"[TranscriptAgent] Processing: {youtube_url}")

        with tempfile.TemporaryDirectory() as tmp_dir:
            audio_path = self._download_audio(youtube_url, tmp_dir)
            transcript = self._transcribe(audio_path)

        print(f"[TranscriptAgent] Done – {len(transcript)} characters transcribed.")
        return transcript

```

```

# -----
# Step 1 – Download
# -----

def _download_audio(self, url: str, output_dir: str) -> str:
    """
    Downloads the audio track of a YouTube video using yt-dlp.

    Picks the smallest available audio format to stay within
    Groq Whisper's 25 MB file-size limit. Saves as .mp3.

    Returns:
        Absolute path to the downloaded audio file.
    """
    output_template = os.path.join(output_dir, "audio.%(ext)s")

    ydl_opts = {
        # Best audio quality but capped at a manageable size.
        # 'worstaudio' keeps files small; for longer videos this
        # matters since Groq Whisper has a 25 MB hard limit.
        "format": "worstaudio/bestaudio",
        "outtmpl": output_template,
        "postprocessors": [
            {
                "key": "FFmpegExtractAudio",
                "preferredcodec": "mp3",
                "preferredquality": "64", # 64 kbps keeps files small
            }
        ],
        # Suppress yt-dlp's own console output so our logs stay clean
        "quiet": True,
        "no_warnings": True,
    }

    print("[TranscriptAgent] Step 1/2 – Downloading audio ...")
    try:
        with yt_dlp.YoutubeDL(ydl_opts) as ydl:
            ydl.download([url])
    except yt_dlp.utils.DownloadError as e:
        raise RuntimeError(f"Audio download failed: {e}") from e

    # Find the file yt-dlp just wrote (extension may vary)
    audio_path = self._find_audio_file(output_dir)

    # Sanity-check file size
    size_mb = os.path.getsize(audio_path) / (1024 * 1024)
    print(f"[TranscriptAgent] Audio downloaded – {size_mb:.1f} MB")

    if size_mb > self.MAX_FILE_SIZE_MB:
        raise RuntimeError(
            f"Audio file is {size_mb:.1f} MB, which exceeds Groq Whisper's "
            f"25 MB limit. Try a shorter video."
        )

    return audio_path

def _find_audio_file(self, directory: str) -> str:
    """Finds the audio file yt-dlp wrote in the given directory."""
    audio_extensions = {".mp3", ".m4a", ".ogg", ".opus", ".wav", ".webm"}
    for filename in os.listdir(directory):
        if os.path.splitext(filename)[1].lower() in audio_extensions:
            return os.path.join(directory, filename)
    raise RuntimeError(
        "Audio download appeared to succeed but no audio file was found. "
        "Check that ffmpeg is installed."
    )

# -----
# Step 2 – Transcribe
# -----

def _transcribe(self, audio_path: str) -> str:
    """

```

```

Sends the audio file to Groq's Whisper API and returns the transcript.

Uses whisper-large-v3-turbo – Groq's fastest Whisper model,
well-suited for English YouTube content.

Returns:
    Plain-text transcript string.
"""
print("[TranscriptAgent] Step 2/2 – Transcribing via Groq Whisper ...")
try:
    with open(audio_path, "rb") as audio_file:
        response = self.client.audio.transcriptions.create(
            file=audio_file,
            model="whisper-large-v3-turbo",
            response_format="text",  # plain string, not JSON
            language="en",
        )
    except Exception as e:
        raise RuntimeError(f"Groq Whisper transcription failed: {e}") from e

    # Groq returns the transcript directly as a string when
    # response_format="text". Clean up any leading/trailing whitespace.
    transcript = str(response).strip()

    if not transcript:
        raise RuntimeError(
            "Whisper returned an empty transcript. "
            "The video may have no speech or be in an unsupported language."
        )

    return transcript

# -----
# Validation
# -----

def _validate_url(self, url: str) -> None:
    """Basic URL sanity check before hitting the network."""
    if not url or not isinstance(url, str):
        raise ValueError("YouTube URL must be a non-empty string.")
    url = url.strip()
    if "youtube.com" not in url and "youtu.be" not in url:
        raise ValueError(
            f"'{url}' does not look like a YouTube URL. "
            "Expected youtube.com oryoutu.be."
        )

# -----
# Standalone test – run this file directly to verify the agent works
# before wiring it into the Orchestrator.
#
#   python agents/transcript_agent.py
# -----

if __name__ == "__main__":
    import sys

    # Short test video (60-second TED-Ed clip – safe for testing)
    TEST_URL = "https://www.youtube.com/watch?v=arj7oStGLkU"

    print("=" * 60)
    print("TranscriptAgent – standalone test")
    print("=" * 60)

    try:
        agent = TranscriptAgent()
        transcript = agent.run(TEST_URL)

        print("\n--- TRANSCRIPT PREVIEW (first 500 chars) ---")
        print(transcript[:500])
        print("..." if len(transcript) > 500 else "")
        print(f"\nTotal length: {len(transcript)} characters")

```

```
    print("\nTranscriptAgent test PASSED")
    sys.exit(0)

except EnvironmentError as e:
    print(f"\nEnvironment error: {e}")
    print("Fix: Add GROQ_API_KEY=your_key_here to a .env file")
    sys.exit(1)

except Exception as e:
    print(f"\nTest FAILED: {e}")
    sys.exit(1)
```

agents/blog_post_agent.py

yt-agent2/agents/blog_post_agent.py

```

"""
BlogPostAgent
-----
Responsibility: Given a plain-text transcript, run an internal
research pass and then write a polished blog post – both steps
using Groq's Llama 3.3 70B model.

This is ONE agent with TWO internal LLM calls.
There is no separate ResearchAgent.

Dependencies: groq, python-dotenv
"""

import os
from groq import Groq
from dotenv import load_dotenv

load_dotenv()

class BlogPostAgent:
    """
    Agent 2 – Researches a transcript internally, then writes a blog post.

    Internal flow:
    1. _research(transcript) -> research_notes (LLM call 1)
    2. _write(transcript, research_notes) -> blog_post (LLM call 2)

    Usage:
    agent = BlogPostAgent()
    blog_post = agent.run(transcript)
    """

    MODEL = "llama-3.3-70b-versatile"

    def __init__(self):
        api_key = os.getenv("GROQ_API_KEY")
        if not api_key:
            raise EnvironmentError(
                "GROQ_API_KEY not found. Add it to your .env file."
            )
        self.client = Groq(api_key=api_key)
        print("[BlogPostAgent] Initialised")

    # -----
    # Public interface
    # -----

    def run(self, transcript: str) -> str:
        """
        Main entry point called by the Orchestrator.

        Args:
            transcript: Plain-text transcript from TranscriptAgent.

        Returns:
            A fully written blog post as a markdown string.

        Raises:
            ValueError: If the transcript is empty.
            RuntimeError: If either LLM call fails.
        """
        self._validate_transcript(transcript)

        print(f"[BlogPostAgent] Received transcript ({len(transcript)} chars)")

        # Step 1 – Internal research pass
        research_notes = self._research(transcript)

```

```

# Step 2 – Write the blog post
blog_post = self._write(transcript, research_notes)

print(f"[BlogPostAgent] Done – blog post written ({len(blog_post)} chars)")
return blog_post

# -----
# Step 1 – Internal research
# -----

def _research(self, transcript: str) -> str:
    """
    First LLM call: analyse the transcript and produce structured
    research notes that will guide the blog post.

    This is an internal step – the notes are never shown to the user.
    """
    print("[BlogPostAgent] Step 1/2 – Running internal research ...")

    system_prompt = """You are a senior content researcher and strategist.
Your job is to deeply analyse a YouTube video transcript and produce
structured research notes that a writer will use to craft a blog post.

Your research notes must include:

1. CORE TOPIC – What is this video fundamentally about? One sentence.

2. TARGET AUDIENCE – Who would benefit most from reading a blog post
about this? Be specific (e.g. "early-stage founders", "patients
recently diagnosed with X", "developers learning Python").

3. KEY INSIGHTS – The 4-6 most valuable, specific ideas from the transcript.
Not summaries – actual insights a reader would find genuinely useful.
Quote or closely paraphrase the most compelling moments.

4. HOOK ANGLE – A compelling opening angle for the blog post. Not a
definition or background. Something that draws the reader in with
a relatable problem, surprising fact, or vivid scenario.

5. SECTION STRUCTURE – Suggest 3-5 section headings that would make
a logical, engaging flow for the blog post. Each heading should
earn the next – no filler sections.

6. HUMAN ELEMENT – Is there a story, analogy, or personal moment in
the transcript that can make the post feel real and relatable?
Identify it specifically.

7. CLOSING DIRECTION – How should the post end? Not a summary.
A forward-looking thought, a call to action, or a reason to care
going forward.

Be specific and concrete. Vague notes produce vague blog posts."""

    user_prompt = f"""Here is the video transcript to analyse:

---
{transcript}
---

Produce your structured research notes now."""

    try:
        response = self.client.chat.completions.create(
            model=self.MODEL,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt},
            ],
            temperature=0.4, # Lower = more focused analysis
            max_tokens=1500,
        )
    except Exception as e:
        raise RuntimeError(f"Research LLM call failed: {e}") from e

```

```

research_notes = response.choices[0].message.content.strip()

print(f"[BlogPostAgent] Research notes ready ({len(research_notes)} chars)")
return research_notes

# -----
# Step 2 – Write the blog post
# -----

def _write(self, transcript: str, research_notes: str) -> str:
    """
    Second LLM call: use the transcript + research notes to write
    a polished, publish-ready blog post.
    """
    print("[BlogPostAgent] Step 2/2 – Writing blog post ...")

    system_prompt = """You are an expert blog writer who produces
publish-ready long-form content. You write in the style of the best
professional blogs – accessible, specific, human, and purposeful.

WRITING PRINCIPLES (follow all of these):

OPENING HOOK
- Never start with a definition or background.
- Open with a relatable problem, a vivid scenario, or a surprising fact
  that makes the reader feel seen before you explain anything.
- The first paragraph should make someone think "this is about me."

STRUCTURE
- Use clear H2 section headings (## in markdown).
- Each section should earn the next – no filler, no repetition.
- 3-5 sections is the right length. Do not pad.

TONE & LANGUAGE
- Write for a general reader, not a specialist.
- Every technical term must be explained in plain words immediately
  after it appears.
- Use short sentences alongside longer ones. Vary the rhythm.
- Be concrete and specific – real examples, not vague generalisations.
- Never write "In conclusion" or "In summary." Just end well.

HUMAN ELEMENT
- Include at least one analogy, story, or vivid real-world example
  that makes an abstract idea feel tangible.
- Write as if you are explaining something fascinating to a smart
  friend, not presenting a report.

ENDING
- Do not summarise what you just said.
- End with a forward-looking thought, a reason to care, or a quiet
  call to action that feels earned.

FORMAT
- Output in clean markdown.
- Start with a # Title
- Use ## for section headings
- No bullet-point dumps – write in flowing paragraphs.
- Target length: 600-900 words (not counting the title).

Do not mention that this is based on a YouTube video or transcript."""

    user_prompt = f"""Here are the research notes for this blog post:

---RESEARCH NOTES---
{research_notes}
---END NOTES---

Here is the original transcript for additional detail and quotes:

---TRANSCRIPT---
{transcript[:3000]}
---END TRANSCRIPT---

Now write the complete, publish-ready blog post in markdown."""

```



```

try:
    response = self.client.chat.completions.create(
        model=self.MODEL,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
        temperature=0.7, # Higher = more creative writing
        max_tokens=2000,
    )
except Exception as e:
    raise RuntimeError(f"Writing LLM call failed: {e}") from e

blog_post = response.choices[0].message.content.strip()

if not blog_post:
    raise RuntimeError("LLM returned an empty blog post.")

return blog_post

# -----
# Validation
# -----

def _validate_transcript(self, transcript: str) -> None:
    if not transcript or not isinstance(transcript, str):
        raise ValueError("Transcript must be a non-empty string.")
    if len(transcript.strip()) < 100:
        raise ValueError(
            f"Transcript is too short ({len(transcript)} chars). "
            "Minimum 100 characters required."
        )

# -----
# Standalone test
# python agents\blog_post_agent.py
# -----

if __name__ == "__main__":
    import sys

    TEST_TRANSCRIPT = """
    So in college, I was a government major, which means I had to write a lot
    of papers. Now, when a normal student writes a paper, they might spread
    their work out a little like this. So, you know, you get started maybe a
    little slowly, but you get enough done in the first week that with some
    heavier days later on, everything gets done and things stay civil.

    And I would want to do that like that. That would be the plan. I would
    have it all ready to go, but then actually the paper would come along,
    and then I would kind of do this. And that would happen every single time.

    So I have a hypothesis that there are two kinds of procrastinators.
    The first type knows what they are. They look at the paper and they say:
    I should be working on this right now, but I'm going to watch YouTube
    instead. The second type doesn't realise how bad they are at it yet.
    They think: I have plenty of time, I'll get to this later.

    The frustrating thing about procrastination isn't that it makes you
    lazy. It's that it makes you feel terrible. There's the Panic Monster –
    that's what wakes you up at 2am when the deadline is tomorrow. Without
    the Panic Monster, nothing would ever get done.

    But here's the problem: the Panic Monster only shows up when there's an
    immediate deadline. What about the procrastination that doesn't have a
    deadline? The big goals – starting a business, writing a book, getting
    healthy, fixing relationships – these have no deadline. And so the Panic
    Monster never appears. And nothing ever happens.

    We all have a finite amount of time. And that's what should scare us more
    than any deadline. The question isn't whether you procrastinate. It's
    whether you're aware of when you're doing it – and whether you're letting
  
```

```
it steal the things that matter most to you.
"""

print("=" * 60)
print("BlogPostAgent – standalone test")
print("=" * 60)

try:
    agent = BlogPostAgent()
    blog_post = agent.run(TEST_TRANSCRIPT)

    print("\n--- BLOG POST OUTPUT ---\n")
    print(blog_post)
    print(f"\nTotal length: {len(blog_post)} characters")
    print("\nBlogPostAgent test PASSED")
    sys.exit(0)

except EnvironmentError as e:
    print(f"\nEnvironment error: {e}")
    print("Fix: Add GROQ_API_KEY=your_key_here to your .env file")
    sys.exit(1)

except Exception as e:
    print(f"\nTest FAILED: {e}")
    sys.exit(1)
```

orchestrator.py

yt-agent2/orchestrator.py

```

"""
Orchestrator
-----
Coordinates TranscriptAgent and BlogPostAgent in sequence.
Receives a YouTube URL, returns a finished blog post.

Flow:
    YouTube URL -> TranscriptAgent -> transcript -> BlogPostAgent -> blog post
"""

from agents.transcript_agent import TranscriptAgent
from agents.blog_post_agent import BlogPostAgent

class Orchestrator:
    """
    Thin coordinator – holds no business logic of its own.
    Just wires the two agents together in the right order.

    Usage:
        orchestrator = Orchestrator()
        blog_post = orchestrator.run("https://www.youtube.com/watch?v=...")
    """

    def __init__(self):
        print("[Orchestrator] Initialising agents ...")
        self.transcript_agent = TranscriptAgent()
        self.blog_post_agent = BlogPostAgent()
        print("[Orchestrator] Ready")

    def run(self, youtube_url: str) -> dict:
        """
        Main entry point called by the Streamlit UI.

        Args:
            youtube_url: Any valid YouTube video URL.

        Returns:
            A dict with keys:
                - 'transcript': the raw transcript string
                - 'blog_post': the finished blog post in markdown
        """
        print(f"\n[Orchestrator] Starting pipeline for: {youtube_url}")

        # Step 1 – Transcribe
        print("[Orchestrator] Calling TranscriptAgent ...")
        transcript = self.transcript_agent.run(youtube_url)
        print(f"[Orchestrator] Transcript received ({len(transcript)} chars)")

        # Step 2 – Write blog post
        print("[Orchestrator] Calling BlogPostAgent ...")
        blog_post = self.blog_post_agent.run(transcript)
        print(f"[Orchestrator] Blog post received ({len(blog_post)} chars)")

        print("[Orchestrator] Pipeline complete")

        return {
            "transcript": transcript,
            "blog_post": blog_post,
        }
# -----

```

```
# Standalone test – tests the full pipeline end to end
# python orchestrator.py
# -----

if __name__ == "__main__":
    import sys

    TEST_URL = "https://www.youtube.com/watch?v=arj7oStGLkU"

    print("=" * 60)
    print("Orchestrator – full pipeline test")
    print("=" * 60)

    try:
        orchestrator = Orchestrator()
        result = orchestrator.run(TEST_URL)

        print("\n--- TRANSCRIPT PREVIEW (first 300 chars) ---")
        print(result["transcript"][:300])

        print("\n--- BLOG POST PREVIEW (first 500 chars) ---")
        print(result["blog_post"][:500])

        print("\nOrchestrator test PASSED – full pipeline working")
        sys.exit(0)

    except Exception as e:
        print(f"\nOrchestrator test FAILED: {e}")
        sys.exit(1)
```

app.pyyt-agent2/app.py

```

import streamlit as st
from orchestrator import Orchestrator

st.set_page_config(
    page_title="YouTube to Blog Post",
    page_icon="[YT]",
    layout="centered"
)

st.title("YouTube to Blog Post")
st.caption("Paste a YouTube URL and get a publish-ready blog post in seconds.")

# -----
# Input
# -----

url = st.text_input(
    label="YouTube URL",
    placeholder="https://www.youtube.com/watch?v=...",
    help="Any public YouTube video with speech will work."
)

generate = st.button("Generate Blog Post", type="primary", use_container_width=True)

# -----
# Pipeline
# -----

if generate:
    if not url.strip():
        st.warning("Please paste a YouTube URL first.")
        st.stop()

    try:
        with st.status("Running pipeline...", expanded=True) as status:

            st.write("Initialising agents...")
            orchestrator = Orchestrator()

            st.write("Downloading and transcribing audio...")
            # Patch TranscriptAgent to report progress via st.write
            original_run = orchestrator.transcript_agent.run

            def run_with_progress(youtube_url):
                result = original_run(youtube_url)
                st.write(f"Transcript ready – {len(result):,} characters")
                return result

            orchestrator.transcript_agent.run = run_with_progress

            st.write("Researching and writing blog post...")
            result = orchestrator.run(url.strip())

            status.update(label="Done!", state="complete", expanded=False)

    except Exception as e:
        st.error(f"Something went wrong: {e}")
        st.stop()

# -----
# Output

```

```
# -----

blog_post = result["blog_post"]
transcript = result["transcript"]

st.divider()

# Blog post tab and transcript tab
tab1, tab2 = st.tabs(["Blog Post", "Raw Transcript"])

with tab1:
    st.markdown(blog_post)
    st.divider()
    st.download_button(
        label="Download as .md",
        data=blog_post,
        file_name="blog_post.md",
        mime="text/markdown",
        use_container_width=True
    )

with tab2:
    st.text_area(
        label="Transcript",
        value=transcript,
        height=400,
        disabled=True
    )
```

requirements.txt

yt-agent2/requirements.txt

```
# YouTube to Blog Post – dependencies
# Install: pip install -r requirements.txt

# Core agent dependencies
yt-dlp>=2024.1.0      # YouTube audio download
groq>=0.9.0           # Groq API client (Whisper + Llama)

# Utilities
python-dotenv>=1.0.0   # Load GROQ_API_KEY from .env

# UI
streamlit>=1.35.0      # Web interface

# ffmpeg is also required by yt-dlp for audio conversion.
# Install separately (not via pip):
#   Ubuntu/Railway: apt-get install -y ffmpeg
#   Mac:           brew install ffmpeg
#   Windows:       https://ffmpeg.org/download.html
```

railway.toml

yt-agent2/railway.toml

```
[build]
builder = "nixpacks"
buildCommand = "pip install -r requirements.txt"

[deploy]
startCommand = "PYTHONPATH=/app streamlit run app.py --server.port $PORT --server.address 0.0.0.0"
```


nixpacks.toml

yt-agent2/nixpacks.toml

```
[phases.setup]
nixPkgs = ["ffmpeg", "python312"]
```